

Computational Geometry Foundations

1 What Is Computational Geometry?

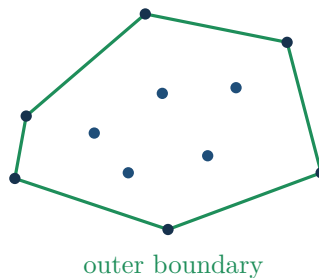
Computational Geometry

Computational geometry studies algorithms for geometric objects such as points, line segments, vectors, polygons, angles, areas, and convex hulls.

The central geometric problem in these notes is:

Convex Hull Motivation

Given a set of points in the plane, find the outer boundary that encloses all of them. This outer boundary is called the **convex hull**.



2 Point Notation

A point in the plane is written as an ordered pair:

$$P = (p_x, p_y).$$

For two points,

$$P = (p_x, p_y), \quad Q = (q_x, q_y).$$

Here p_x, q_x are x -coordinates, and p_y, q_y are y -coordinates.

3 Vectors Between Points

Vector from P to Q

The vector from P to Q is obtained by subtracting coordinates:

$$\overrightarrow{PQ} = Q - P = (q_x - p_x, q_y - p_y).$$

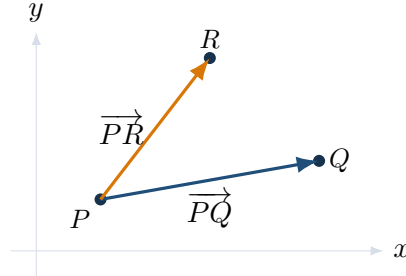
For three points

$$P = (p_x, p_y), \quad Q = (q_x, q_y), \quad R = (r_x, r_y),$$

the two vectors starting at P are

$$\overrightarrow{PQ} = (q_x - p_x, q_y - p_y),$$

$$\overrightarrow{PR} = (r_x - p_x, r_y - p_y).$$



These vectors decide whether the path

$$P \rightarrow Q \rightarrow R$$

turns left, turns right, or stays straight.

4 2D Cross Product

The 2D cross product of vectors

$$\vec{a} = (a_x, a_y), \quad \vec{b} = (b_x, b_y)$$

is the scalar

$$\vec{a} \times \vec{b} = a_x b_y - a_y b_x.$$

For points P, Q, R , use

$$\vec{a} = \overrightarrow{PQ}, \quad \vec{b} = \overrightarrow{PR}.$$

5 Orientation Formula

Orientation Test

For

$$P = (p_x, p_y), \quad Q = (q_x, q_y), \quad R = (r_x, r_y),$$

define

$$\text{orient}(P, Q, R) = (q_x - p_x)(r_y - p_y) - (q_y - p_y)(r_x - p_x).$$

Equivalently,

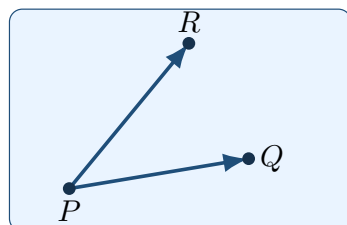
$$\text{orient}(P, Q, R) = \overrightarrow{PQ} \times \overrightarrow{PR}.$$

In determinant form:

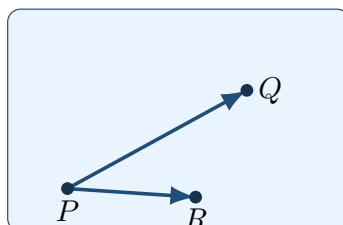
$$\overrightarrow{PQ} \times \overrightarrow{PR} = \begin{vmatrix} q_x - p_x & q_y - p_y \\ r_x - p_x & r_y - p_y \end{vmatrix}.$$

6 Meaning of the Sign

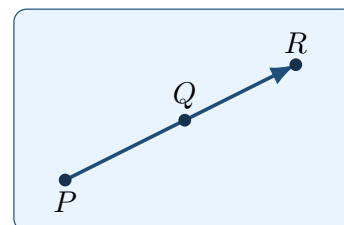
Value of $\text{orient}(P, Q, R)$	Meaning
Positive	Left turn, counterclockwise orientation.
Negative	Right turn, clockwise orientation.
Zero	Collinear points.



$\text{orient} > 0$: left turn



$\text{orient} < 0$: right turn



$\text{orient} = 0$: collinear

7 Worked Examples

Each example uses the same two-step method:

$$\overrightarrow{PQ} = Q - P, \quad \overrightarrow{PR} = R - P,$$

then

$$\text{orient}(P, Q, R) = \overrightarrow{PQ} \times \overrightarrow{PR}.$$

7.1 Left Turn

Example 1

$$P = (1, 1), \quad Q = (4, 2), \quad R = (2, 5).$$

$$\overrightarrow{PQ} = (4 - 1, 2 - 1) = (3, 1),$$

$$\overrightarrow{PR} = (2 - 1, 5 - 1) = (1, 4).$$

Now compute:

$$\text{orient}(P, Q, R) = 3 \cdot 4 - 1 \cdot 1 = 12 - 1 = 11.$$

Since

$$11 > 0,$$

the movement from P to Q to R is counterclockwise.

Result

$$\text{orient}(P, Q, R) > 0 \Rightarrow \text{left turn.}$$

7.2 Right Turn

Example 2

$$P = (1, 1), \quad Q = (2, 5), \quad R = (4, 2).$$

$$\overrightarrow{PQ} = (2 - 1, 5 - 1) = (1, 4),$$

$$\overrightarrow{PR} = (4 - 1, 2 - 1) = (3, 1).$$

$$\text{orient}(P, Q, R) = 1 \cdot 1 - 4 \cdot 3 = 1 - 12 = -11.$$

Since

$$-11 < 0,$$

the movement from P to Q to R is clockwise.

Result

$$\text{orient}(P, Q, R) < 0 \quad \Rightarrow \quad \text{right turn.}$$

7.3 Collinear Points

Example 3

$$P = (0, 0), \quad Q = (2, 2), \quad R = (4, 4).$$

$$\overrightarrow{PQ} = (2, 2), \quad \overrightarrow{PR} = (4, 4).$$

$$\text{orient}(P, Q, R) = 2 \cdot 4 - 2 \cdot 4 = 8 - 8 = 0.$$

Result

$$\text{orient}(P, Q, R) = 0 \quad \Rightarrow \quad P, Q, R \text{ are collinear.}$$

7.4 Quick Comparison

Case	Points	orient	Meaning
Left turn	$P = (1, 1), Q = (4, 2), R = (2, 5)$	11	counterclockwise
Right turn	$P = (1, 1), Q = (2, 5), R = (4, 2)$	-11	clockwise
Collinear	$P = (0, 0), Q = (2, 2), R = (4, 4)$	0	same straight line

8 Why Orientation Is Useful

The orientation test is preferred over angle measurement because it:

- avoids trigonometric functions,
- is fast to compute,
- is exact for integer coordinates when integer arithmetic is used,
- is the basic test behind convex hull algorithms.

Implementation Warning

For large integer coordinates, the products

$$(q_x - p_x)(r_y - p_y) \quad \text{and} \quad (q_y - p_y)(r_x - p_x)$$

may overflow fixed-size integer types. Use a sufficiently large numeric type.

9 Final Review

Foundations Summary

$P = (p_x, p_y)$ represents a point in the plane.

$\overrightarrow{PQ} = (q_x - p_x, q_y - p_y)$.

$\text{orient}(P, Q, R) = \overrightarrow{PQ} \times \overrightarrow{PR}$.

$\text{orient} > 0$: left turn; $\text{orient} < 0$: right turn; $\text{orient} = 0$: collinear.

Orientation Examples

1 Orientation Formula Review

Orientation Test

For three points

$$P = (p_x, p_y), \quad Q = (q_x, q_y), \quad R = (r_x, r_y),$$

$$\text{orient}(P, Q, R) = (q_x - p_x)(r_y - p_y) - (q_y - p_y)(r_x - p_x).$$

The sign tells the turn direction:

Sign	Meaning
$\text{orient} > 0$	Left turn, counterclockwise.
$\text{orient} < 0$	Right turn, clockwise.
$\text{orient} = 0$	Collinear points.

2 Left-Turn Example

Problem

Let

$$P = (1, 1), \quad Q = (4, 2), \quad R = (5, 5).$$

Determine the turn made by

$$P \rightarrow Q \rightarrow R.$$

Solution

First compute the two vectors from P :

$$\overrightarrow{PQ} = (4 - 1, 2 - 1) = (3, 1),$$

$$\overrightarrow{PR} = (5 - 1, 5 - 1) = (4, 4).$$

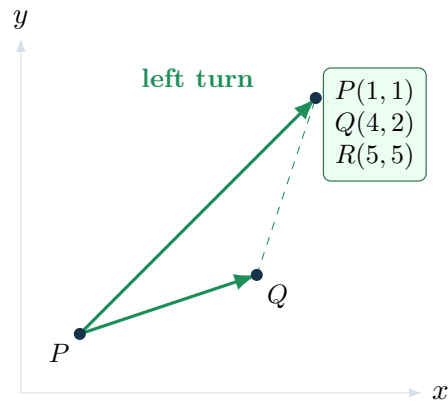
Now compute the cross product:

$$\text{orient}(P, Q, R) = 3(4) - 1(4) = 12 - 4 = 8.$$

Since

$$8 > 0,$$

the points form a left turn.



Answer

$\text{orient}(P, Q, R) = 8 > 0$, so the turn is left.

3 Right-Turn Example

Problem

Let

$$P = (1, 1), \quad Q = (4, 4), \quad R = (5, 2).$$

Determine the turn made by

$$P \rightarrow Q \rightarrow R.$$

Solution

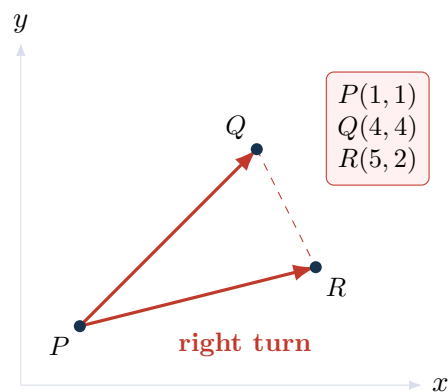
$$\begin{aligned} \overrightarrow{PQ} &= (4 - 1, 4 - 1) = (3, 3), \\ \overrightarrow{PR} &= (5 - 1, 2 - 1) = (4, 1). \end{aligned}$$

$$\text{orient}(P, Q, R) = 3(1) - 3(4) = 3 - 12 = -9.$$

Since

$$-9 < 0,$$

the points form a right turn.



Answer

$$\text{orient}(P, Q, R) = -9 < 0, \quad \text{so the turn is right.}$$

4 Collinear Points

Collinearity Test

If

$$\text{orient}(P, Q, R) = 0,$$

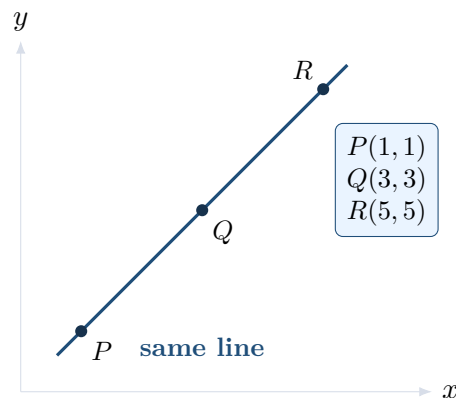
then P, Q, R lie on the same straight line.

Example:

$$P = (1, 1), \quad Q = (3, 3), \quad R = (5, 5).$$

$$\overrightarrow{PQ} = (2, 2), \quad \overrightarrow{PR} = (4, 4).$$

$$\text{orient}(P, Q, R) = 2(4) - 2(4) = 8 - 8 = 0.$$



5 Collinear Rule for Convex Hulls

When several points have the same direction from a pivot point, their orientation with the pivot is zero. In convex hull algorithms, the usual rule is:

Keep the Farther Point

If two candidate points have the same polar angle from the pivot, keep the farther point and discard the nearer point.

Let the pivot be

$$P_0 = (0, 0),$$

and let

$$A = (2, 2), \quad B = (4, 4).$$

Both points lie on the same ray from P_0 , so

$$\text{orient}(P_0, A, B) = 0.$$

Compare squared distances from P_0 :

$$d^2(P_0, A) = 2^2 + 2^2 = 8,$$

$$d^2(P_0, B) = 4^2 + 4^2 = 32.$$

Since

$$32 > 8,$$

keep B and discard A .

Why Discard the Nearer Point?

The nearer point lies inside the same boundary direction. Keeping it may place an interior collinear point on the convex hull, which is usually not desired.

6 Final Review

Orientation Examples Summary

$\text{orient} > 0 \Rightarrow$ left turn.
 $\text{orient} < 0 \Rightarrow$ right turn.
 $\text{orient} = 0 \Rightarrow$ collinear.
For same-angle hull candidates, keep the farther point.

Line Segment Intersection

1 Problem Definition

Segment Intersection

Given two line segments

$$\overline{P_1P_2} \quad \text{and} \quad \overline{P_3P_4},$$

decide whether the two segments intersect.

The output is a Boolean answer:

`true` or `false`.

Two segments intersect if they share at least one point. This includes:

- a proper crossing;
- touching at an endpoint;
- overlapping along a collinear interval.

2 Line Segment as a Convex Combination

Line Segment

For two endpoints

$$A = (a_x, a_y), \quad B = (b_x, b_y),$$

the segment $\overline{AB}$ is the set of all points

$$X = \alpha A + (1 - \alpha)B, \quad 0 \leq \alpha \leq 1.$$

The condition $0 \leq \alpha \leq 1$ means X lies between A and B , not outside the segment.

3 Orientation Review

For three points

$$A = (a_x, a_y), \quad B = (b_x, b_y), \quad C = (c_x, c_y),$$

the orientation test is

$$\text{orient}(A, B, C) = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x).$$

Value	Meaning
$\text{orient}(A, B, C) > 0$	$A \rightarrow B \rightarrow C$ makes a left turn.
$\text{orient}(A, B, C) < 0$	$A \rightarrow B \rightarrow C$ makes a right turn.
$\text{orient}(A, B, C) = 0$	A, B, C are collinear.

Sign Convention

These notes use

$$\text{orient}(A, B, C) = \overrightarrow{AB} \times \overrightarrow{AC}.$$

Some textbooks define a procedure called DIRECTION using the reversed cross-product order. That reverses the sign. In this note, always use the sign table above.

4 Main Idea: Straddling

Two segments properly intersect when each segment crosses the infinite line containing the other segment.

For

$$\overline{P_1P_2} \quad \text{and} \quad \overline{P_3P_4},$$

compute four orientation values:

$$\begin{aligned} d_1 &= \text{orient}(P_3, P_4, P_1), & d_2 &= \text{orient}(P_3, P_4, P_2), \\ d_3 &= \text{orient}(P_1, P_2, P_3), & d_4 &= \text{orient}(P_1, P_2, P_4). \end{aligned}$$

General Intersection Rule

$$d_1d_2 < 0 \quad \text{and} \quad d_3d_4 < 0 \quad \implies \quad \text{segments properly intersect.}$$

The condition $d_1d_2 < 0$ means P_1 and P_2 are on opposite sides of the line through P_3P_4 . The condition $d_3d_4 < 0$ means P_3 and P_4 are on opposite sides of the line through P_1P_2 .

5 Boundary Case: On Segment

If an orientation value is zero, then three points are collinear. But collinearity alone is not enough. The point must also lie between the segment endpoints.

On-Segment Test

Assume A, B, C are collinear. Then C lies on segment $\overline{AB}$ if

$$\min(a_x, b_x) \leq c_x \leq \max(a_x, b_x)$$

and

$$\min(a_y, b_y) \leq c_y \leq \max(a_y, b_y).$$

6 Complete Algorithm

Segments Intersect

```
SegmentsIntersect(p1, p2, p3, p4):
    d1 = orientation(p3, p4, p1)
    d2 = orientation(p3, p4, p2)
    d3 = orientation(p1, p2, p3)
```

```

d4 = orientation(p1, p2, p4)

if d1 * d2 < 0 and d3 * d4 < 0:
    return true

if d1 == 0 and OnSegment(p3, p4, p1):
    return true

if d2 == 0 and OnSegment(p3, p4, p2):
    return true

if d3 == 0 and OnSegment(p1, p2, p3):
    return true

if d4 == 0 and OnSegment(p1, p2, p4):
    return true

return false

OnSegment(a, b, c):
    return min(ax, bx) <= cx <= max(ax, bx)
           and min(ay, by) <= cy <= max(ay, by)

```

7 Example 1: Proper Crossing

Problem

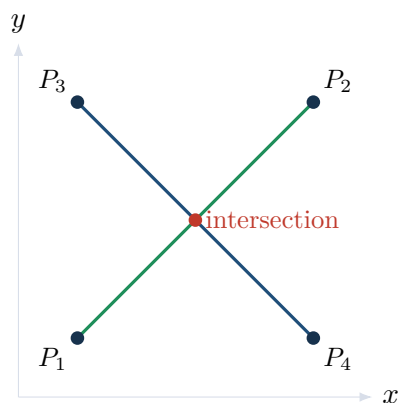
Test whether

$$P_1 = (1, 1), \quad P_2 = (5, 5)$$

and

$$P_3 = (1, 5), \quad P_4 = (5, 1)$$

intersect.



Compute:

$$d_1 = \text{orient}(P_3, P_4, P_1) = (5 - 1)(1 - 5) - (1 - 5)(1 - 1) = 4(-4) - (-4)(0) = -16.$$

$$d_2 = \text{orient}(P_3, P_4, P_2) = (5 - 1)(5 - 5) - (1 - 5)(5 - 1) = 4(0) - (-4)(4) = 16.$$

$$d_3 = \text{orient}(P_1, P_2, P_3) = (5-1)(5-1) - (5-1)(1-1) = 4(4) - 4(0) = 16.$$

$$d_4 = \text{orient}(P_1, P_2, P_4) = (5-1)(1-1) - (5-1)(5-1) = 4(0) - 4(4) = -16.$$

Now:

$$d_1 d_2 = (-16)(16) < 0, \quad d_3 d_4 = (16)(-16) < 0.$$

Answer

The two segments properly intersect.

8 Example 2: No Intersection

Problem

Test whether

$$P_1 = (1, 1), \quad P_2 = (5, 1)$$

and

$$P_3 = (2, 3), \quad P_4 = (4, 4)$$

intersect.

Use the line through $P_1 P_2$:

$$d_3 = \text{orient}(P_1, P_2, P_3) = (5-1)(3-1) - (1-1)(2-1) = 8.$$

$$d_4 = \text{orient}(P_1, P_2, P_4) = (5-1)(4-1) - (1-1)(4-1) = 12.$$

Both d_3 and d_4 are positive, so P_3 and P_4 are on the same side of the line through $P_1 P_2$. Therefore the second segment does not straddle the first line.

Answer

The two segments do not intersect.

9 Example 3: Endpoint on Segment

Problem

Test whether

$$P_1 = (1, 1), \quad P_2 = (5, 1)$$

and

$$P_3 = (3, 1), \quad P_4 = (3, 4)$$

intersect.

Point $P_3 = (3, 1)$ lies on the horizontal segment $\overline{P_1 P_2}$. Verify this using orientation:

$$d_3 = \text{orient}(P_1, P_2, P_3) = (5-1)(1-1) - (1-1)(3-1) = 0.$$

Since $d_3 = 0$, the points P_1, P_2, P_3 are collinear. Now apply the on-segment test:

$$\min(1, 5) \leq 3 \leq \max(1, 5), \quad \min(1, 1) \leq 1 \leq \max(1, 1).$$

That is,

$$1 \leq 3 \leq 5, \quad 1 \leq 1 \leq 1.$$

Both are true, so P_3 lies on $\overline{P_1P_2}$.

Answer

The two segments intersect at $P_3 = (3, 1)$.

10 Example 4: Collinear but Disjoint

Problem

Test whether

$$P_1 = (1, 1), \quad P_2 = (3, 1)$$

and

$$P_3 = (4, 1), \quad P_4 = (6, 1)$$

intersect.

All four points lie on the same horizontal line. For example:

$$\text{orient}(P_1, P_2, P_3) = (3 - 1)(1 - 1) - (1 - 1)(4 - 1) = 0.$$

But $P_3 = (4, 1)$ is not on segment $\overline{P_1P_2}$, because

$$\min(1, 3) \leq 4 \leq \max(1, 3)$$

becomes

$$1 \leq 4 \leq 3,$$

which is false.

Also, $P_2 = (3, 1)$ is not on segment $\overline{P_3P_4}$, because

$$4 \leq 3 \leq 6$$

is false.

Answer

The segments are collinear but disjoint.

11 Example 5: Overlapping Collinear Segments

Problem

Test whether

$$P_1 = (1, 1), \quad P_2 = (5, 1)$$

and

$$P_3 = (3, 1), \quad P_4 = (7, 1)$$

intersect.

The segments are collinear:

$$\text{orient}(P_1, P_2, P_3) = 0, \quad \text{orient}(P_1, P_2, P_4) = 0.$$

Now check whether an endpoint of one segment lies on the other segment. For $P_3 = (3, 1)$:

$$\min(1, 5) \leq 3 \leq \max(1, 5), \quad \min(1, 1) \leq 1 \leq \max(1, 1).$$

So:

$$1 \leq 3 \leq 5, \quad 1 \leq 1 \leq 1.$$

Both are true, so P_3 lies on $\overline{P_1P_2}$. Therefore the segments overlap.

Answer

The segments intersect along the overlap from $(3, 1)$ to $(5, 1)$.

12 Complexity

The algorithm computes four orientation values and a constant number of range checks.

Time Complexity

$$\mathcal{O}(1)$$

13 Final Review

Segment Intersection Summary

$d_1 = \text{orient}(P_3, P_4, P_1), \quad d_2 = \text{orient}(P_3, P_4, P_2).$
 $d_3 = \text{orient}(P_1, P_2, P_3), \quad d_4 = \text{orient}(P_1, P_2, P_4).$
 $d_1d_2 < 0$ and $d_3d_4 < 0 \Rightarrow$ proper intersection.
 $d_i = 0 \Rightarrow$ use the on-segment boundary check.
The full test runs in $\mathcal{O}(1)$.

Convex Hull

1 Problem Definition

Convex Hull

Given a finite set of planar points

$$S = \{P_1, P_2, \dots, P_n\},$$

the **convex hull** of S is the smallest convex set that contains every point of S . When the points are not all collinear, this set is a convex polygon.

The input is a set of points:

$$S = \{P_1, P_2, \dots, P_n\}.$$

The output is an ordered list of boundary vertices:

$$H = (H_1, H_2, \dots, H_m), \quad m \leq n.$$

The vertices $H_1, \dots, H_m$ are the outermost points. Points strictly inside the polygon are not hull vertices.

Rubber-Band Interpretation

Imagine stretching a rubber band around all the points and then releasing it. The rubber band touches the outer boundary points. Those touched points form the convex hull.

2 Why It Is Called “Convex”

A set is convex if the line segment between any two points in the set stays inside the set. The convex hull is the smallest convex shape that contains the original point set.

Smallest Convex Container

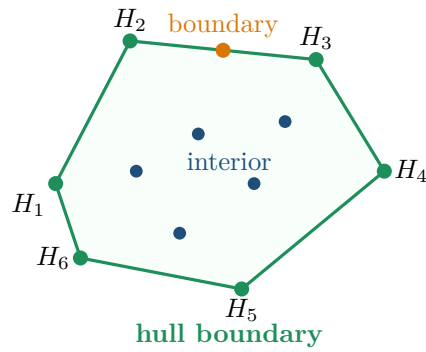
$$\text{CH}(S) = \text{the smallest convex region containing } S.$$

For algorithm problems, we usually return only the boundary vertices of this region, not every point inside the region.

3 Hull Vertices, Boundary Points, and Interior Points

For a point set, a point may be:

Type	Meaning
Hull vertex	A corner point of the outer boundary.
Boundary point	A point on an edge of the hull but not necessarily a corner.
Interior point	A point strictly inside the hull; not part of the hull vertex list.



Boundary Convention

Some algorithms return every point on the boundary, including collinear edge points. Many convex-hull algorithms return only the corner vertices. In these notes, “hull vertices” means the corner points unless stated otherwise.

4 Convex Polygon

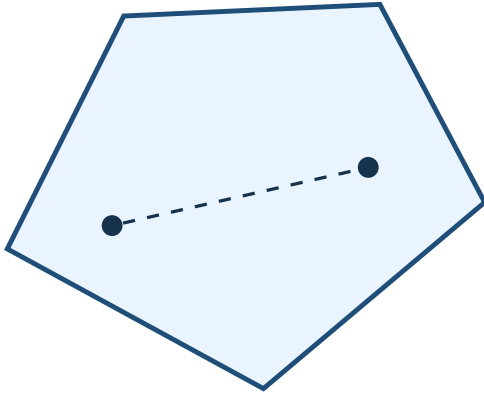
Convex Polygon

A polygon is **convex** if it does not bend inward.

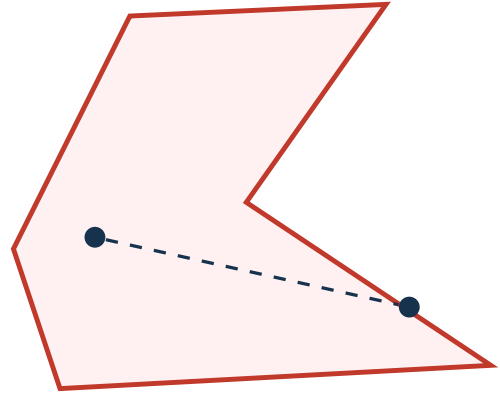
Equivalent tests for convexity:

- every interior angle is at most 180° ;
- for a strictly convex polygon, every interior angle is less than 180° ;
- every line segment joining two points inside the polygon stays inside the polygon;
- when the boundary is traversed in one direction, there are no opposite-sign turns; for a strictly convex polygon, all turns have the same nonzero orientation.

5 Convex vs Non-Convex



convex polygon
segment stays inside



non-convex polygon
segment leaves polygon

6 Orientation View of the Hull

Convex hull algorithms depend heavily on orientation tests. For three points

$$A = (A_x, A_y), \quad B = (B_x, B_y), \quad C = (C_x, C_y),$$

the orientation value is

$$\text{orient}(A, B, C) = (B_x - A_x)(C_y - A_y) - (B_y - A_y)(C_x - A_x).$$

Orientation Test

$$\begin{aligned} \text{orient}(A, B, C) > 0 &\implies \text{left turn,} \\ \text{orient}(A, B, C) < 0 &\implies \text{right turn,} \\ \text{orient}(A, B, C) = 0 &\implies \text{collinear points.} \end{aligned}$$

If the hull vertices are listed counterclockwise as

$$H_1, H_2, \dots, H_m,$$

then every consecutive boundary turn should be a left turn:

$$\text{orient}(H_i, H_{i+1}, H_{i+2}) > 0,$$

where indices wrap around the boundary.

Same-Turn Rule

A convex polygon has no inward turn when traversed around its boundary.

For a clockwise hull, all nonzero turns should be right turns. If a turn has the opposite sign from the traversal direction, the polygon bends inward at that point.

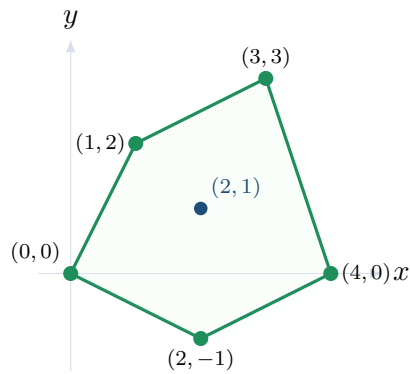
7 Example 1: One Interior Point

Problem

Given

$$S = \{(0, 0), (1, 2), (2, 1), (3, 3), (4, 0), (2, -1)\},$$

identify the hull vertices.



The point $(2, 1)$ lies inside the polygon, so it is not a hull vertex.

Answer

One clockwise ordering of the convex hull is

$$(0, 0), (1, 2), (3, 3), (4, 0), (2, -1).$$

8 Example 2: All Points Collinear

Problem

Find the convex hull of

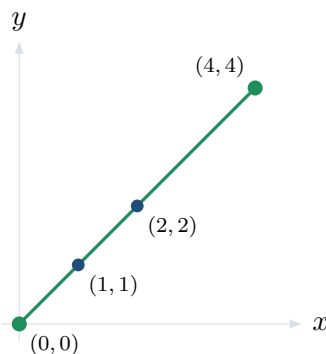
$$S = \{(0, 0), (1, 1), (2, 2), (4, 4)\}.$$

All points lie on the same line:

$$y = x.$$

For example,

$$\text{orient}((0, 0), (1, 1), (4, 4)) = 1 \cdot 4 - 1 \cdot 4 = 0.$$



Answer

If only extreme hull vertices are required, the hull is the line segment

$$(0, 0) \rightarrow (4, 4).$$

The middle points are boundary points but not corner vertices.

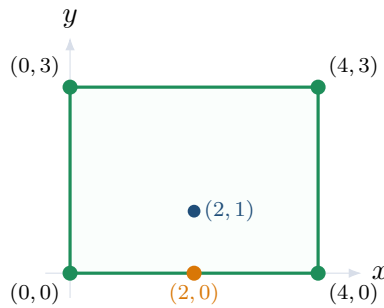
9 Example 3: Boundary Collinear Point

Problem

Consider

$$S = \{(0, 0), (2, 0), (4, 0), (4, 3), (0, 3), (2, 1)\}.$$

Which points are hull vertices?



The point $(2, 0)$ lies on the bottom edge. It is on the boundary, but it is not a corner. The point $(2, 1)$ is strictly inside.

Answer

The hull vertices are

$$(0, 0), (4, 0), (4, 3), (0, 3).$$

10 Orientation Check on a Hull

Use the hull from Example 1:

$$H_1 = (0, 0), \quad H_2 = (1, 2), \quad H_3 = (3, 3).$$

Check the first turn:

$$\overrightarrow{H_1 H_2} = (1, 2), \quad \overrightarrow{H_1 H_3} = (3, 3).$$

$$\text{orient}(H_1, H_2, H_3) = 1 \cdot 3 - 2 \cdot 3 = 3 - 6 = -3.$$

This value is negative because the listed order in Example 1 is clockwise, not counterclockwise. The same-turn rule still applies: a clockwise hull should have consistently negative turns.

Ordering Matters

The hull may be listed clockwise or counterclockwise. What matters is consistency: all boundary turns should have the same sign for a convex polygon.

11 Final Review

Convex Hull Summary

The convex hull is the smallest convex set containing all points.
Hull vertices are outermost corner points.
Interior points are excluded from the hull vertex list.
Boundary collinear points may be excluded depending on convention.
A valid convex boundary has consistent orientation signs.

Graham Scan

1 Overview

Graham Scan

Graham Scan computes the convex hull by sorting points around a pivot and then using a stack to remove points that create invalid turns.

The algorithm has two main ideas:

- Sort points in angular order around a guaranteed hull point.
- Scan the sorted points and remove any point that creates a right turn or an unwanted collinear point.

Core Intuition

While walking around the outside boundary, the hull must keep turning in one consistent direction. In counterclockwise order, the valid turn is a left turn.

2 Input and Output

Input:

$$S = \{P_1, P_2, \dots, P_n\}.$$

Output:

$$H = (H_1, H_2, \dots, H_m), \quad m \leq n,$$

where H is the convex hull boundary in order.

3 Main Steps

1. Choose a pivot point P_0 .
2. Sort all other points by polar angle around P_0 .
3. If multiple points have the same angle, keep the farthest one when only corner vertices are required.
4. Push the first few sorted points onto a stack.
5. For each next point, use orientation to test the turn.
6. Pop stack points while the turn is invalid.
7. Return the stack as the hull.

4 Choosing the Pivot

Choose the point with the smallest y -coordinate. If there is a tie, choose the point with the smallest x -coordinate.

$$P_0 = \text{point with minimum } y, \text{ breaking ties by minimum } x.$$

This point is guaranteed to lie on the convex hull because no point is below it.

5 Polar-Angle Sorting

Sort all other points counterclockwise by their polar angle around P_0 .

Squared Distance

If two points have the same polar angle from P_0 , compare squared distances:

$$d^2(P_0, Q) = (q_x - p_x)^2 + (q_y - p_y)^2.$$

Keep the farther point and discard the nearer point if the hull output should contain only corner vertices.

Squared distance is enough because the square root does not change the ordering.

Boundary Convention

Keeping only the farthest point for each polar angle removes points that lie on the same ray from the pivot. This matches the convention that the convex hull output contains only corner vertices. If an implementation must return every boundary point, same-angle and collinear cases need a different tie-handling rule.

6 Stack Rule

Suppose the top two stack points are A and B , and the next scanned point is C . Compute:

$$\text{orient}(A, B, C).$$

For

$$A = (A_x, A_y), \quad B = (B_x, B_y), \quad C = (C_x, C_y),$$

the orientation value is

$$\text{orient}(A, B, C) = (B_x - A_x)(C_y - A_y) - (B_y - A_y)(C_x - A_x).$$

Orientation value	Stack action
$\text{orient}(A, B, C) > 0$	Left turn. Keep B , then push C .
$\text{orient}(A, B, C) < 0$	Right turn. Pop B .
$\text{orient}(A, B, C) = 0$	Collinear. Usually keep the farther point and discard the nearer point.

Stack Invariant

After each scan step, the stack stores the current outer boundary candidate in correct order.

7 Pseudocode

Graham Scan

```
GrahamScan(points):
    remove duplicate points

    if number of points <= 2:
        return points

    p0 = point with minimum y-coordinate
        breaking ties by minimum x-coordinate

    sort remaining points by polar angle around p0

    for points with the same polar angle:
        keep only the farthest point from p0

    if number of remaining sorted points == 0:
        return [p0]

    if number of remaining sorted points == 1:
        return [p0, first sorted point]

    stack = empty stack
    push p0
    push first sorted point
    push second sorted point

    for each remaining point p:
        while stack has at least two points
            and orientation(next_to_top, top, p) <= 0:
                pop stack

        push p

    return stack
```

This version removes right turns and interior collinear points.

8 Correctness Proof

Let the points after pivot selection and polar-angle sorting be

$$P_0, P_1, P_2, \dots, P_m,$$

where P_0 is the pivot and $P_1, \dots, P_m$ are sorted counterclockwise by polar angle.

Loop Invariant

At the start of each iteration that processes P_i , the stack S , read from bottom to top, contains exactly the vertices of

$$\text{CH}(\{P_0, P_1, \dots, P_{i-1}\})$$

in counterclockwise order.

Initialization

Before the loop starts,

$$S = [P_0, P_1, P_2].$$

These are exactly the hull vertices of the first three processed points, so the invariant is true.

Maintenance

Assume the invariant is true before processing P_i . Let

$$A = \text{NextToTop}(S), \quad B = \text{Top}(S).$$

The algorithm tests the turn

$$\text{orient}(A, B, P_i).$$

If the turn is not left, then B lies inside the new boundary or on one of its edges, so B cannot remain a hull corner and is popped. The while-loop removes all such invalid top points. When a left turn is finally obtained, pushing P_i gives the hull of

$$\{P_0, P_1, \dots, P_i\}$$

in counterclockwise order. Thus the invariant is maintained.

Termination

When the loop ends, all points have been processed. By the invariant, S contains exactly the vertices of

$$\text{CH}(\{P_0, P_1, \dots, P_m\}).$$

Therefore Graham Scan returns the convex hull vertices in counterclockwise order.

Correctness Result

Graham Scan returns exactly the convex hull vertices in counterclockwise order.

9 Worked Example

Point Set

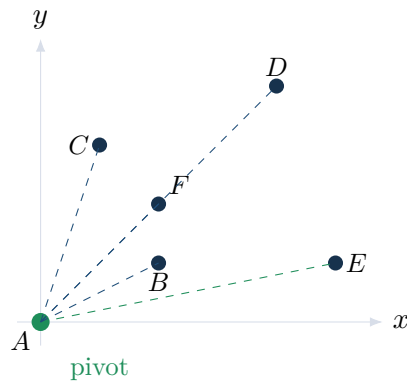
$$A = (0, 0), \quad B = (2, 1), \quad C = (1, 3),$$

$$D = (4, 4), \quad E = (5, 1), \quad F = (2, 2).$$

The pivot is

$$A = (0, 0)$$

because it has the smallest y -coordinate.



10 Polar-Angle Order

Around pivot A , the points have the following angle order:

$$\tan \theta = \frac{y - y_A}{x - x_A}.$$

Since $A = (0, 0)$, this becomes

$$\tan \theta = \frac{y}{x}, \quad \theta = \tan^{-1}\left(\frac{y}{x}\right).$$

Point	Vector from A	Polar angle
$E = (5, 1)$	$(5, 1)$	11.31°
$B = (2, 1)$	$(2, 1)$	26.565°
$F = (2, 2)$	$(2, 2)$	45°
$D = (4, 4)$	$(4, 4)$	45°
$C = (1, 3)$	$(1, 3)$	71.565°

Initial order:

$$A, E, B, F, D, C.$$

Since F and D have the same polar angle, compare distances:

$$AF^2 = 2^2 + 2^2 = 8, \quad AD^2 = 4^2 + 4^2 = 32.$$

Since

$$32 > 8,$$

keep D and discard F .

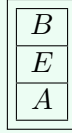
Effective Scan Order

$$A, E, B, D, C.$$

11 Scan Trace

Stack Notation

In this trace, the stack is drawn vertically with the top item in the highest cell:



Here A is at the bottom, B is at the top, and the next orientation test uses the top two points.

11.1 Start with A, E, B

Push:

A, E, B .

So the stack becomes



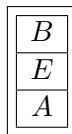
Check:

$\text{orient}(A, E, B)$.

$$\overrightarrow{AE} = (5, 1), \quad \overrightarrow{AB} = (2, 1).$$

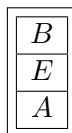
$$\text{orient}(A, E, B) = 5(1) - 1(2) = 5 - 2 = 3.$$

Since $3 > 0$, this is a left turn. Keep B . The stack remains



11.2 Process D

The next point is D . Before inserting D , the stack is



Check:

$\text{orient}(E, B, D)$.

$$\overrightarrow{EB} = (2 - 5, 1 - 1) = (-3, 0),$$

$$\overrightarrow{ED} = (4 - 5, 4 - 1) = (-1, 3).$$

$$\text{orient}(E, B, D) = (-3)(3) - 0(-1) = -9.$$

Since $-9 < 0$, this is a right turn. Pop B .

After popping B , the stack becomes



Now check:

$$\text{orient}(A, E, D).$$

$$\text{orient}(A, E, D) = 5(4) - 1(4) = 20 - 4 = 16.$$

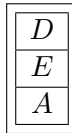
Since $16 > 0$, push D .

After pushing D , the stack becomes



11.3 Process C

The next point is C . Before inserting C , the stack is



Check:

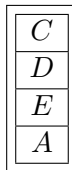
$$\text{orient}(E, D, C).$$

$$\overrightarrow{ED} = (-1, 3), \quad \overrightarrow{EC} = (1 - 5, 3 - 1) = (-4, 2).$$

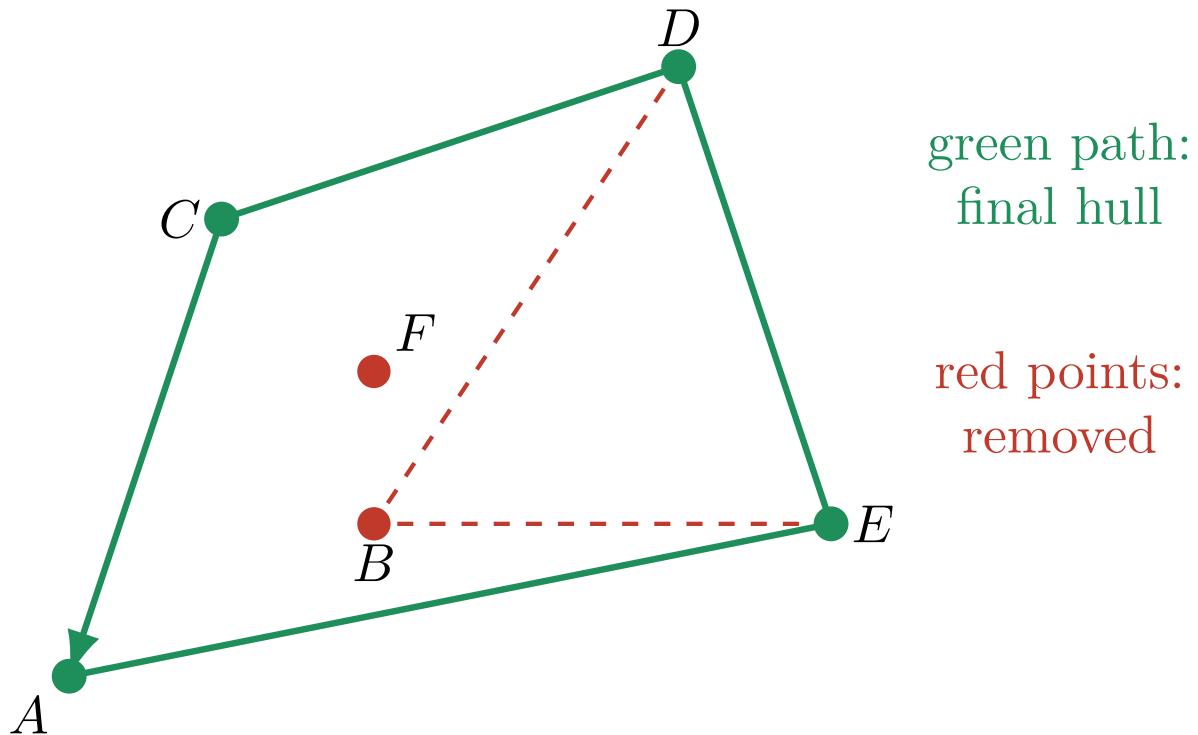
$$\text{orient}(E, D, C) = (-1)(2) - 3(-4) = -2 + 12 = 10.$$

Since $10 > 0$, push C .

After pushing C , the final stack is



12 Visual Trace



13 Final Hull

The final stack is:

top $\rightarrow C, D, E, A.$

So the convex hull in boundary order is:

$A \rightarrow E \rightarrow D \rightarrow C \rightarrow A.$

The removed points are:

B and $F.$

14 Second Worked Example

Problem

Apply Graham Scan to

$$P_0 = (0, 0), \quad P_1 = (1, 1), \quad P_2 = (2, 0), \quad P_3 = (2, 2), \quad P_4 = (0, 2).$$

The pivot is already

$$P_0 = (0, 0),$$

because it has the smallest y -coordinate and then the smallest x -coordinate.

Sorted Order

Since $P_0 = (0, 0)$, each polar angle can be found from

$$\tan \theta = \frac{y}{x}.$$

Point	Angle calculation	Polar angle
$P_2 = (2, 0)$	$\tan \theta = \frac{0}{2} = 0$	0°
$P_1 = (1, 1)$	$\tan \theta = \frac{1}{1} = 1$	45°
$P_3 = (2, 2)$	$\tan \theta = \frac{2}{2} = 1$	45°
$P_4 = (0, 2)$	vertical ray from P_0	90°

Therefore the initial polar-angle order around P_0 is

$$\boxed{P_2, P_1, P_3, P_4.}$$

Points P_1 and P_3 have the same polar angle 45° . Compare squared distances:

$$d^2(P_0, P_1) = 1^2 + 1^2 = 2,$$

$$d^2(P_0, P_3) = 2^2 + 2^2 = 8.$$

Since P_3 is farther, discard P_1 . The point P_1 lies on the line segment from P_0 to P_3 , so it is not a corner vertex.

Effective scan order:

$$\boxed{P_0, P_2, P_3, P_4.}$$

Scan

Start with

$$P_0, P_2, P_3.$$

The stack becomes

$$\boxed{\begin{array}{c} P_3 \\ P_2 \\ P_0 \end{array}}$$

Check:

$$\text{orient}(P_0, P_2, P_3).$$

$$\overrightarrow{P_0P_2} = (2, 0), \quad \overrightarrow{P_0P_3} = (2, 2).$$

$$\text{orient}(P_0, P_2, P_3) = 2(2) - 0(2) = 4 > 0.$$

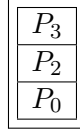
So this is a left turn. Keep P_3 , and the stack stays

$$\boxed{\begin{array}{c} P_3 \\ P_2 \\ P_0 \end{array}}$$

Now process P_4 :

$$\text{orient}(P_2, P_3, P_4).$$

Before inserting P_4 , the stack is

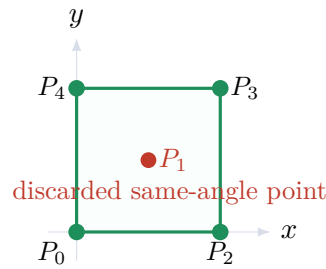
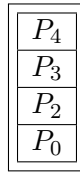


$$\overrightarrow{P_2P_3} = (0, 2), \quad \overrightarrow{P_2P_4} = (-2, 2).$$

$$\text{orient}(P_2, P_3, P_4) = 0(2) - 2(-2) = 4 > 0.$$

So this is also a left turn. Push P_4 .

The final stack is



Second Example Result

$$P_0 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_0$$

is the convex hull. Point P_1 is discarded because it is nearer than P_3 on the same ray from the pivot.

15 Complexity

Choosing the pivot takes $\mathcal{O}(n)$. Sorting the remaining points by polar angle takes

$$\mathcal{O}(n \log n).$$

The stack scan takes $\mathcal{O}(n)$, because each point is pushed once and popped at most once.

Time Complexity

$$\mathcal{O}(n) + \mathcal{O}(n \log n) + \mathcal{O}(n) = \mathcal{O}(n \log n).$$

16 Final Review

Graham Scan Summary

Choose pivot P_0 with minimum y , then minimum x .
Sort all other points by polar angle around P_0 .
For equal angles, keep the farthest point.
Use a stack and pop while orientation ≤ 0 .
Running time is $\mathcal{O}(n \log n)$.

Jarvis March

1 Overview

Jarvis March

Jarvis March, also called the **gift-wrapping algorithm**, computes the convex hull by repeatedly choosing the next outermost point.

The algorithm behaves like wrapping a string around the outside of the point set:

- start from an extreme point;
- choose the next hull vertex;
- continue walking around the boundary;
- stop when the walk returns to the starting point.

Main Idea

At each hull vertex, choose the next point so that all other points lie on the same side of the directed edge.

2 Input and Output

Input:

$$S = \{P_1, P_2, \dots, P_n\}.$$

Output:

$$H = (H_1, H_2, \dots, H_h),$$

where H is the convex hull boundary and h is the number of hull vertices.

3 Algorithm Steps

1. Select the leftmost point P_0 . This means choose the point with minimum x -coordinate. If there is a tie, choose the point with minimum y -coordinate:

$$P_0 = \text{point with minimum } x, \text{ breaking ties by minimum } y.$$

This point is guaranteed to be on the convex hull because no point lies farther left than it.

2. Set current = P_0 .
3. Add current to the hull list.
4. Choose any point different from current as the temporary candidate.
5. Scan every other point R in the input set.

6. Compare R with the current candidate using

$$\text{orient}(\text{current}, \text{candidate}, R).$$

7. If R is more outside than the current candidate, replace the candidate:

$$\text{candidate} = R.$$

8. Continue scanning using the updated candidate.

9. After all points have been checked, the final candidate is the next hull vertex.

10. Set $\text{current} = \text{candidate}$ and repeat.

11. Stop when $\text{current} = P_0$, because the hull has wrapped back to the start.

Candidate Update Rule

For each hull point, Jarvis March does not choose the next vertex immediately. It keeps a temporary candidate, updates it while scanning all points, and chooses the final remaining candidate only after the full scan is finished.

4 Choosing the Next Vertex

Suppose the current hull point is P , the current candidate is Q , and another point being tested is R . Compute

$$\text{orient}(P, Q, R).$$

For

$$P = (P_x, P_y), \quad Q = (Q_x, Q_y), \quad R = (R_x, R_y),$$

the orientation value is

$$\text{orient}(P, Q, R) = (Q_x - P_x)(R_y - P_y) - (Q_y - P_y)(R_x - P_x).$$

Case	Decision
$\text{orient}(P, Q, R) < 0$	R is more clockwise than Q . Replace Q by R .
$\text{orient}(P, Q, R) > 0$	Keep Q .
$\text{orient}(P, Q, R) = 0$	Points are collinear. Keep the farther point from P .

Collinear Tie Rule

If Q and R are collinear from P , choose the point with larger squared distance:

$$d^2(P, R) > d^2(P, Q) \implies \text{choose } R.$$

5 Pseudocode

Jarvis March

```
JarvisMarch(points):  
    if number of points < 3:  
        return points  
  
    start = leftmost point  
    hull = empty list  
    current = start  
  
    repeat:  
        add current to hull  
        candidate = any point different from current  
  
        for each point r:  
            turn = orientation(current, candidate, r)  
  
            if r is more clockwise than candidate:  
                candidate = r  
  
            else if turn == 0 and r is farther from current:  
                candidate = r  
  
        current = candidate  
  
    until current == start  
  
    return hull
```

6 Worked Example

Problem

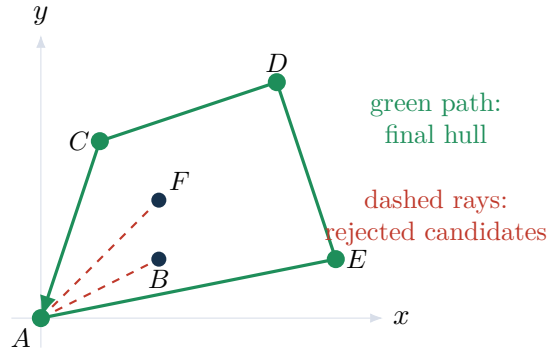
Apply Jarvis March to

$$\begin{aligned} A &= (0, 0), & B &= (2, 1), & C &= (1, 3), \\ D &= (4, 4), & E &= (5, 1), & F &= (2, 2). \end{aligned}$$

Step 1: Choose the Starting Point

The starting point is the leftmost point. If there is a tie, choose the lower one.

$\text{start} = A = (0, 0).$



Step 2: Find the Next Point from A

Set

$$\text{current} = A.$$

For this example, we wrap clockwise. Therefore:

$$\text{orient}(\text{current}, \text{candidate}, R) < 0 \implies R \text{ replaces the candidate.}$$

Start with B as a temporary candidate:

$$\text{candidate} = B.$$

The first useful replacement happens when E is tested. The vectors are

$$\overrightarrow{AB} = (2, 1), \quad \overrightarrow{AE} = (5, 1).$$

$$\text{orient}(A, B, E) = 2(1) - 1(5) = 2 - 5 = -3.$$

Since the value is negative, E is more clockwise than B . Replace the candidate:

$$\text{candidate} = E.$$

The full candidate trace from A is:

Test point	Orientation value	Candidate after test
C	$\text{orient}(A, B, C) = 5 > 0$	Keep B .
D	$\text{orient}(A, B, D) = 4 > 0$	Keep B .
E	$\text{orient}(A, B, E) = -3 < 0$	Replace by E .
F	$\text{orient}(A, E, F) = 8 > 0$	Keep E .

So the first hull edge is

$$A \rightarrow E.$$

The hull list is now

$$H = [A, E].$$

Step 3: Continue Wrapping

Repeat the same rule from the newest hull point. Each table shows how the temporary candidate changes while scanning the other points.

From E

Start with A as a temporary candidate.

Test point	Orientation value	Candidate after test
B	$\text{orient}(E, A, B) = -3 < 0$	Replace by B .
C	$\text{orient}(E, B, C) = -6 < 0$	Replace by C .
D	$\text{orient}(E, C, D) = -10 < 0$	Replace by D .
F	$\text{orient}(E, D, F) = 8 > 0$	Keep D .

So the next hull edge is

$$E \rightarrow D.$$

From D

Start with A as a temporary candidate.

Test point	Orientation value	Candidate after test
B	$\text{orient}(D, A, B) = 4 > 0$	Keep A .
C	$\text{orient}(D, A, C) = -8 < 0$	Replace by C .
E	$\text{orient}(D, C, E) = 10 > 0$	Keep C .
F	$\text{orient}(D, C, F) = 4 > 0$	Keep C .

So the next hull edge is

$$D \rightarrow C.$$

From C

Start with A as a temporary candidate.

Test point	Orientation value	Candidate after test
B	$\text{orient}(C, A, B) = 5 > 0$	Keep A .
D	$\text{orient}(C, A, D) = 8 > 0$	Keep A .
E	$\text{orient}(C, A, E) = 14 > 0$	Keep A .
F	$\text{orient}(C, A, F) = 4 > 0$	Keep A .

So the next hull edge is

$$C \rightarrow A.$$

The algorithm stops when the next point is the starting point A .

Worked Example Result

$$A \rightarrow E \rightarrow D \rightarrow C \rightarrow A$$

is the convex hull. The points B and F are not hull vertices.

7 Second Example: Few Hull Points

Problem

Find the hull of

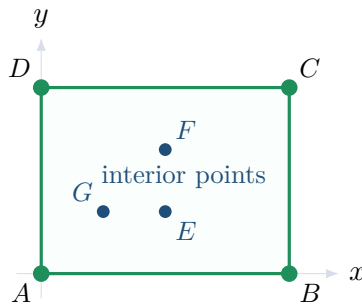
$$S = \{(0, 0), (4, 0), (4, 3), (0, 3), (2, 1), (2, 2), (1, 1)\}.$$

Name the points as

$$A = (0, 0), \quad B = (4, 0), \quad C = (4, 3), \quad D = (0, 3),$$

$$E = (2, 1), \quad F = (2, 2), \quad G = (1, 1).$$

The points E, F, G are inside the rectangle. Jarvis March still scans them, but they never remain as the final candidate.



Step 1: Start at A

The leftmost point is

$$A = (0, 0).$$

So A is the starting hull point.

Step 2: Choose the Next Point from A

Start with B as the temporary candidate. Now scan the remaining points.

Test point	Orientation value	Candidate after test
C	$\text{orient}(A, B, C) = 4(3) - 0(4) = 12 > 0$	Keep B .
D	$\text{orient}(A, B, D) = 4(3) - 0(0) = 12 > 0$	Keep B .
E	$\text{orient}(A, B, E) = 4(1) - 0(2) = 4 > 0$	Keep B .
F	$\text{orient}(A, B, F) = 4(2) - 0(2) = 8 > 0$	Keep B .
G	$\text{orient}(A, B, G) = 4(1) - 0(1) = 4 > 0$	Keep B .

After all points are checked, the final candidate is B . Therefore:

$$A \rightarrow B.$$

Step 3: Choose the Next Point from B

Start with A as the temporary candidate. Testing C gives a negative orientation, so C replaces A :

$$\text{orient}(B, A, C) = (-4)(3) - 0(0) = -12 < 0.$$

Now continue scanning using C as the candidate.

Test point	Orientation value	Candidate after test
D	$\text{orient}(B, C, D) = 0(3) - 3(-4) = 12 > 0$	Keep C .
E	$\text{orient}(B, C, E) = 0(1) - 3(-2) = 6 > 0$	Keep C .
F	$\text{orient}(B, C, F) = 0(2) - 3(-2) = 6 > 0$	Keep C .
G	$\text{orient}(B, C, G) = 0(1) - 3(-3) = 9 > 0$	Keep C .

After all points are checked, the final candidate is C . Therefore:

$$\boxed{B \rightarrow C.}$$

Step 4: Choose the Next Point from C

Start with A as the temporary candidate. Point B does not replace A :

$$\text{orient}(C, A, B) = (-4)(-3) - (-3)(0) = 12 > 0.$$

Testing D gives

$$\text{orient}(C, A, D) = (-4)(0) - (-3)(-4) = -12 < 0.$$

So D replaces A . Continue scanning using D as the candidate.

Testing the interior points gives:

$$\text{orient}(C, D, E) = (-4)(-2) - 0(-2) = 8 > 0,$$

$$\text{orient}(C, D, F) = (-4)(-1) - 0(-2) = 4 > 0,$$

$$\text{orient}(C, D, G) = (-4)(-2) - 0(-3) = 8 > 0.$$

All three values are positive, so E, F, G do not replace D .

After all points are checked, the final candidate is D . Therefore:

$$\boxed{C \rightarrow D.}$$

Step 5: Choose the Next Point from D

Start with A as the temporary candidate. Every remaining point stays on the left side of $D \rightarrow A$, so A remains the candidate.

Test point	Orientation value	Candidate after test
B	$\text{orient}(D, A, B) = 0(-3) - (-3)(4) = 12 > 0$	Keep A .
C	$\text{orient}(D, A, C) = 0(0) - (-3)(4) = 12 > 0$	Keep A .
E	$\text{orient}(D, A, E) = 0(-2) - (-3)(2) = 6 > 0$	Keep A .
F	$\text{orient}(D, A, F) = 0(-1) - (-3)(2) = 6 > 0$	Keep A .
G	$\text{orient}(D, A, G) = 0(-2) - (-3)(1) = 3 > 0$	Keep A .

The next point is the starting point A , so the algorithm stops.

Second Example Result

$$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$$

That is,

$$(0, 0) \rightarrow (4, 0) \rightarrow (4, 3) \rightarrow (0, 3) \rightarrow (0, 0)$$

is the convex hull. The interior points E, F, G are checked during the scan, but none of them remains as the final candidate.

8 Complexity

Let:

$$n = \text{number of input points}, \quad h = \text{number of hull vertices}.$$

For each hull vertex, Jarvis March scans all n points to find the next hull vertex. Therefore:

$$\text{time complexity} = \mathcal{O}(nh).$$

The algorithm is output-sensitive because the running time depends on h , the number of hull vertices.

When Jarvis March Is Good

Jarvis March is especially useful when the number of hull vertices h is small compared with the total number of points n .

9 Graham Scan vs Jarvis March

Feature	Graham Scan	Jarvis March
Main idea	Sort by angle, then use a stack.	Wrap around the outside.
Starting point	Lowest point.	Leftmost point.
Key operation	Orientation while scanning stack.	Orientation while choosing next hull point.
Time complexity	$\mathcal{O}(n \log n)$.	$\mathcal{O}(nh)$.
Best when	General use, many hull points.	Few hull points.

10 Final Review

Jarvis March Summary

Start at the leftmost point.
Repeatedly choose the next outermost point.
Use orientation to compare candidates.
For collinear candidates, choose the farther point.
Stop when the start point is reached again.
Running time is $\mathcal{O}(nh)$.

Point Inside a Convex Polygon

1 Problem Definition

Point-In-Convex-Polygon Test

Given a convex polygon and a query point Q , decide whether Q is inside the polygon, outside the polygon, or on its boundary.

Assume the polygon vertices are listed in boundary order:

$$P_1, P_2, \dots, P_n.$$

Each edge is directed from

$$P_i \rightarrow P_{i+1},$$

where the last vertex wraps back to the first:

$$P_{n+1} = P_1.$$

2 Same-Sign Orientation Principle

For every directed edge $P_i P_{i+1}$, compute

$$s_i = \text{orient}(P_i, P_{i+1}, Q).$$

The value s_i tells which side of edge $P_i P_{i+1}$ the query point lies on.

Decision Rule

all $s_i > 0$	$\Rightarrow Q$ is inside for counterclockwise order,
all $s_i < 0$	$\Rightarrow Q$ is inside for clockwise order,
mixed signs	$\Rightarrow Q$ is outside,
$s_i = 0$	$\Rightarrow Q$ may be on an edge.

Core Idea

For a convex polygon, an inside point stays on the same side of every boundary edge.

3 Boundary Check

If

$$\text{orient}(P_i, P_{i+1}, Q) = 0,$$

then P_i, P_{i+1}, Q are collinear. The point is on the segment only if its coordinates lie within the endpoint ranges.

On-Segment Test

For edge AB , point Q lies on segment AB when

$$\min(a_x, b_x) \leq q_x \leq \max(a_x, b_x)$$

and

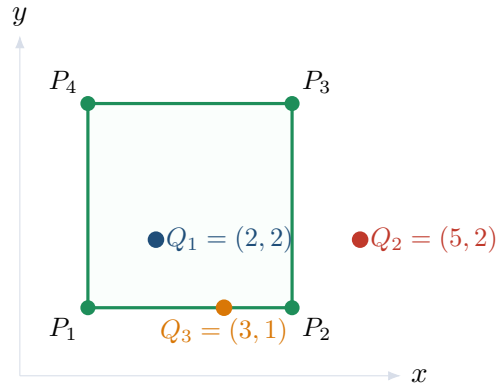
$$\min(a_y, b_y) \leq q_y \leq \max(a_y, b_y).$$

4 Example Polygon

Use the square:

$$P_1 = (1, 1), \quad P_2 = (4, 1), \quad P_3 = (4, 4), \quad P_4 = (1, 4).$$

The vertices are ordered counterclockwise.



5 Inside-Point Example

Problem

Test whether

$$Q = (2, 2)$$

is inside the square.

Solution

For edge P_1P_2 :

$$\overrightarrow{P_1P_2} = (3, 0), \quad \overrightarrow{P_1Q} = (1, 1).$$

$$\text{orient}(P_1, P_2, Q) = 3(1) - 0(1) = 3.$$

For edge P_2P_3 :

$$\overrightarrow{P_2P_3} = (0, 3), \quad \overrightarrow{P_2Q} = (-2, 1).$$

$$\text{orient}(P_2, P_3, Q) = 0(1) - 3(-2) = 6.$$

For edge P_3P_4 :

$$\overrightarrow{P_3P_4} = (-3, 0), \quad \overrightarrow{P_3Q} = (-2, -2). \\ \text{orient}(P_3, P_4, Q) = (-3)(-2) - 0(-2) = 6.$$

For edge P_4P_1 :

$$\overrightarrow{P_4P_1} = (0, -3), \quad \overrightarrow{P_4Q} = (1, -2). \\ \text{orient}(P_4, P_1, Q) = 0(-2) - (-3)(1) = 3.$$

All signs are positive:

$$3, 6, 6, 3.$$

Answer

$Q = (2, 2)$ is inside the square.

6 Outside-Point Example

Problem

Test whether

$$Q = (5, 2)$$

is inside the same square.

For edge P_1P_2 :

$$\overrightarrow{P_1P_2} = (3, 0), \quad \overrightarrow{P_1Q} = (4, 1). \\ \text{orient}(P_1, P_2, Q) = 3(1) - 0(4) = 3.$$

For edge P_2P_3 :

$$\overrightarrow{P_2P_3} = (0, 3), \quad \overrightarrow{P_2Q} = (1, 1). \\ \text{orient}(P_2, P_3, Q) = 0(1) - 3(1) = -3.$$

The signs are already mixed:

$$3, -3.$$

Answer

$Q = (5, 2)$ is outside the square.

7 Boundary Example

Problem

Test whether

$$Q = (3, 1)$$

is inside, outside, or on the boundary of the same square.

For edge P_1P_2 :

$$\overrightarrow{P_1P_2} = (3, 0), \quad \overrightarrow{P_1Q} = (2, 0). \\ \text{orient}(P_1, P_2, Q) = 3(0) - 0(2) = 0.$$

So Q is collinear with P_1P_2 . Collinearity alone only says that Q lies on the same infinite line as the edge. To be on the actual edge segment, Q must also lie between the two endpoints.

Here the edge endpoints are

$$P_1 = (1, 1), \quad P_2 = (4, 1).$$

For a point $Q = (q_x, q_y)$ to lie on the segment P_1P_2 , its coordinates must satisfy the bounding-box test:

$$\min(1, 4) \leq q_x \leq \max(1, 4), \quad \min(1, 1) \leq q_y \leq \max(1, 1).$$

Since $Q = (3, 1)$, this becomes:

$$1 \leq 3 \leq 4, \quad 1 \leq 1 \leq 1.$$

Both conditions are true, so Q lies between P_1 and P_2 , not merely on the same line.

Answer

$Q = (3, 1)$ lies on the boundary edge P_1P_2 .

8 Pseudocode

Point in Convex Polygon

```
PointInConvexPolygon(polygon, q):
    positive = false
    negative = false

    for each directed edge (pi, pj):
        value = orientation(pi, pj, q)

        if value > 0:
            positive = true

        else if value < 0:
            negative = true

        else:
            if q lies on segment pi-pj:
                return "boundary"

    if positive and negative:
        return "outside"

    return "inside"
```

9 Correctness Intuition

For a convex polygon, every boundary edge acts like a separating line. If Q is inside, then Q remains on the same side of every directed boundary edge. If one edge puts Q on the opposite side, then that edge separates Q from the polygon interior, so Q is outside.

Convexity Requirement

The same-sign orientation test works directly for convex polygons. It should not be applied unchanged to arbitrary non-convex polygons.

10 Complexity

The algorithm checks each polygon edge once. For n polygon vertices, there are n edges.

time complexity = $\mathcal{O}(n)$.

Only a few variables are needed:

auxiliary space = $\mathcal{O}(1)$.

11 Final Review

Point-In-Convex-Polygon Summary

$s_i = \text{orient}(P_i, P_{i+1}, Q)$.
All positive or all negative signs mean inside.
Mixed signs mean outside.
 $s_i = 0$ requires an on-segment boundary check.
The method runs in $\mathcal{O}(n)$ time for a convex polygon.

Area Methods in Computational Geometry

1 Overview

Area Methods

Area methods use coordinate formulas to solve two common computational-geometry tasks:

- compute the area of a polygon;
- decide whether a query point is inside, outside, or on the boundary of a polygon.

Method	Input	Output
Triangle area	Three points A, B, C	Area of $\triangle ABC$.
Shoelace formula	Polygon vertices in boundary order	Polygon area.
Area-sum method	Polygon and query point Q	Inside, outside, or boundary decision.

Recommended Order

For point-in-polygon by area, first compute the polygon area using Shoelace. Then compute the sum of triangle areas formed by the query point and each polygon edge.

2 Base Formula: Triangle Area

For three points

$$A = (x_1, y_1), \quad B = (x_2, y_2), \quad C = (x_3, y_3),$$

the triangle area is:

Triangle Area

$$\text{Area}(\triangle ABC) = \frac{1}{2} |x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)|.$$

Equivalently, using orientation:

$$\text{Area}(\triangle ABC) = \frac{1}{2} |\text{orient}(A, B, C)|.$$

Example

Problem

Find the area of

$$A = (1, 1), \quad B = (4, 1), \quad C = (2, 3).$$

$$\overrightarrow{AB} = (4 - 1, 1 - 1) = (3, 0), \quad \overrightarrow{AC} = (2 - 1, 3 - 1) = (1, 2).$$

$$\text{orient}(A, B, C) = 3(2) - 0(1) = 6.$$

$$\text{Area}(\triangle ABC) = \frac{1}{2}|6| = 3.$$

Answer

$$\text{Area}(\triangle ABC) = 3 \text{ square units.}$$

3 Method 1: Shoelace Formula

Use Shoelace when the polygon vertices are known in clockwise or counterclockwise boundary order:

$$P_0 = (x_0, y_0), \quad P_1 = (x_1, y_1), \quad \dots, \quad P_{n-1} = (x_{n-1}, y_{n-1}).$$

Use wraparound:

$$P_n = P_0.$$

Shoelace Formula

$$A = \frac{1}{2} \left| \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) \right|.$$

Vertex Order

The vertices must be listed around the boundary. A random order of the same points does not give the correct polygon area.

4 Example 1: Shoelace Formula

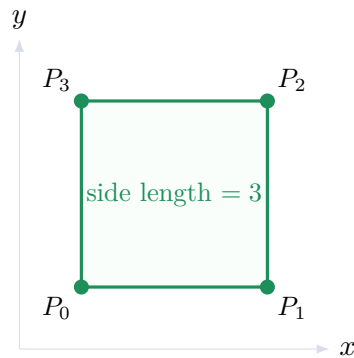
Problem

Find the area of the square

$$P_0 = (1, 1), \quad P_1 = (4, 1), \quad P_2 = (4, 4), \quad P_3 = (1, 4).$$

Solution

The vertices are already listed in counterclockwise boundary order. Draw the square:



Use

$$A = \frac{1}{2} \left| \sum_{i=0}^3 (x_i y_{i+1} - x_{i+1} y_i) \right|, \quad P_4 = P_0.$$

Edge	Shoelace term	Value
$P_0 P_1$	$x_0 y_1 - x_1 y_0 = 1(1) - 4(1)$	-3
$P_1 P_2$	$x_1 y_2 - x_2 y_1 = 4(4) - 4(1)$	12
$P_2 P_3$	$x_2 y_3 - x_3 y_2 = 4(4) - 1(4)$	12
$P_3 P_0$	$x_3 y_0 - x_0 y_3 = 1(1) - 1(4)$	-3

So the signed double-area is

$$-3 + 12 + 12 - 3 = 18.$$

Therefore:

$$A = \frac{1}{2} |18| = 9.$$

Answer

Area = 9 square units.

5 Example 2: Shoelace Table Form

The same calculation can be done using the table form. Write the first point again at the bottom:

Point	x_i	y_i
P_0	1	1
P_1	4	1
P_2	4	4
P_3	1	4
P_0	1	1

Add the downward diagonal products:

$$1 \cdot 1 + 4 \cdot 4 + 4 \cdot 4 + 1 \cdot 1 = 1 + 16 + 16 + 1 = 34.$$

Add the upward diagonal products:

$$1 \cdot 4 + 1 \cdot 4 + 4 \cdot 1 + 4 \cdot 1 = 4 + 4 + 4 + 4 = 16.$$

Then:

$$A = \frac{1}{2}|34 - 16| = \frac{1}{2}(18) = 9.$$

Table Form Result

The table form gives the same area: 9.

6 Method 2: Area-Sum Point Test

Let the polygon be

$$P_0P_1 \cdots P_{n-1},$$

and let Q be the query point. Connect Q to every polygon edge:

$$\triangle QP_0P_1, \quad \triangle QP_1P_2, \quad \dots, \quad \triangle QP_{n-1}P_0.$$

Compute

$$S = \sum_{i=0}^{n-1} \text{Area}(\triangle QP_iP_{i+1}), \quad P_n = P_0.$$

Let A be the polygon area. Then:

Area-Sum Decision Rule

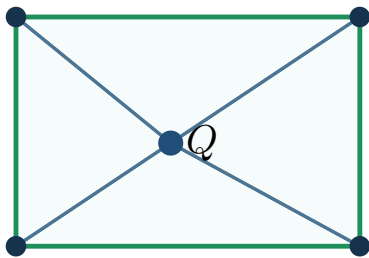
$$\begin{aligned} S = A &\implies Q \text{ is inside or on the boundary,} \\ S > A &\implies Q \text{ is outside.} \end{aligned}$$

If $S = A$ and one of the triangle areas is 0, then Q lies on a boundary edge. If $S = A$ and no triangle area is 0, then Q is strictly inside.

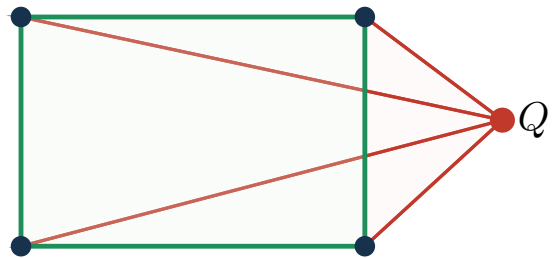
7 Why Area-Sum Works

If Q is inside the polygon, the triangles from Q to each edge exactly partition the polygon. Their areas add up to the polygon area.

If Q is outside, the triangles overlap or include extra region outside the polygon. Then the area sum becomes larger than the polygon area.



inside: triangles fill polygon



outside: extra area is counted

Numerical Tolerance

In numerical programs, avoid checking $S = A$ using exact floating-point equality. Use

$$|S - A| \leq \varepsilon.$$

8 Example 3: Inside Point

Problem

Use the same square and test

$$Q = (2, 2).$$

Solution

The polygon area from the shoelace formula is

$$A = 9.$$

Now split the polygon into four triangles:

$$\triangle QP_0P_1, \quad \triangle QP_1P_2, \quad \triangle QP_2P_3, \quad \triangle QP_3P_0.$$

Compute each triangle area using

$$\text{Area}(\triangle ABC) = \frac{1}{2} |\text{orient}(A, B, C)|.$$

Triangle	Orientation calculation	Area
$\triangle QP_0P_1$	$(1 - 2)(1 - 2) - (1 - 2)(4 - 2) = (-1)(-1) - (-1)(2) = 3$	$\frac{3}{2}$
$\triangle QP_1P_2$	$(4 - 2)(4 - 2) - (1 - 2)(4 - 2) = 2(2) - (-1)(2) = 6$	3
$\triangle QP_2P_3$	$(4 - 2)(4 - 2) - (4 - 2)(1 - 2) = 2(2) - 2(-1) = 6$	3
$\triangle QP_3P_0$	$(1 - 2)(1 - 2) - (4 - 2)(1 - 2) = (-1)(-1) - 2(-1) = 3$	$\frac{3}{2}$

Therefore:

$$S = \frac{3}{2} + 3 + 3 + \frac{3}{2} = 9.$$

Since

$$S = A = 9,$$

the point is inside or on the boundary. Since no triangle area is zero, Q is strictly inside.

Answer

$Q = (2, 2)$ is inside the square.

9 Example 4: Outside Point

Problem

Use the same square and test

$$R = (5, 2).$$

Solution

Again, the polygon area is

$$A = 9.$$

Compute the four triangle areas:

$$\text{Area}(\triangle RP_0P_1) = \frac{1}{2}|(-4)(-1) - (-1)(-1)| = \frac{3}{2},$$

$$\text{Area}(\triangle RP_1P_2) = \frac{1}{2}|(-1)(2) - (-1)(-1)| = \frac{3}{2},$$

$$\text{Area}(\triangle RP_2P_3) = \frac{1}{2}|(-1)(2) - (2)(-4)| = 3,$$

$$\text{Area}(\triangle RP_3P_0) = \frac{1}{2}|(-4)(-1) - (2)(-4)| = 6.$$

So:

$$S = \frac{3}{2} + \frac{3}{2} + 3 + 6 = 12.$$

Since

$$S = 12 > A = 9,$$

the triangle areas count extra region outside the square.

Answer

$R = (5, 2)$ is outside the square.

10 Example 5: Boundary Point

Problem

Use the same square and test

$$T = (3, 1).$$

Solution

The point T lies on the bottom edge P_0P_1 . The triangle using that edge has zero area:

$$\text{Area}(\triangle TP_0P_1) = \frac{1}{2}|\text{orient}(T, P_0, P_1)| = 0.$$

The remaining triangle areas still fill the square:

$$\text{Area}(\triangle TP_1P_2) = \frac{3}{2},$$

$$\text{Area}(\triangle TP_2P_3) = \frac{9}{2},$$

$$\text{Area}(\triangle TP_3P_0) = 3.$$

Therefore:

$$S = 0 + \frac{3}{2} + \frac{9}{2} + 3 = 9 = A.$$

Answer

$T = (3, 1)$ is on the boundary of the square.

11 Complexity

Both methods scan polygon edges once.

Method	Time complexity
Area-sum inclusion test	$\mathcal{O}(n)$ for n polygon edges.
Shoelace area formula	$\mathcal{O}(n)$ for n polygon vertices.

12 Final Review

Area Methods Summary

$$S = \sum \text{Area}(\triangle QP_iP_{i+1}).$$

$$S = A \Rightarrow Q \text{ is inside or on boundary}; \quad S > A \Rightarrow Q \text{ is outside.}$$

$$A = \frac{1}{2} \left| \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i) \right|.$$

Use $P_n = P_0$ for wraparound.